GALTOSM Appendix

Shounak Saha, Susmoy Das and Arpit Sharma

Table of Contents

Contents

1 Workflow of GALTOSM **1**

1.1 CADP to STORM/PRISM 1

1.2 mCRL2 to STORM/PRISM 1

1.3 PRISM/STORM to CADP 1

2 Grammar of Logics **2**

2.0.1 APCTL 2

2.0.2 APCTL* 3

2.0.3 APRCTL 4

2.0.4 PCTL 5

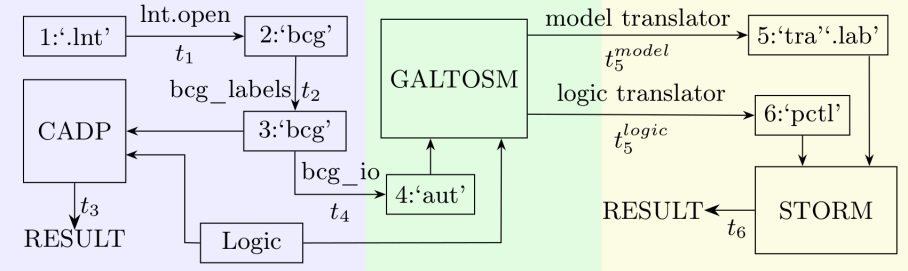
2.0.5 PCTL* 5

2.0.6 PRCTL 6

1 Workflow of GALTOSM

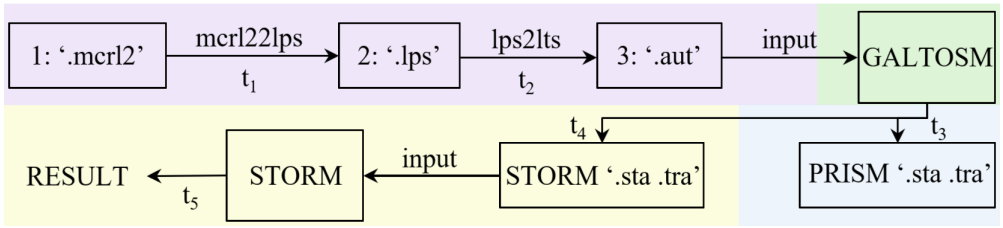
1.1 CADP to STORM/PRISM

The workflow is explained in details in the main paper.



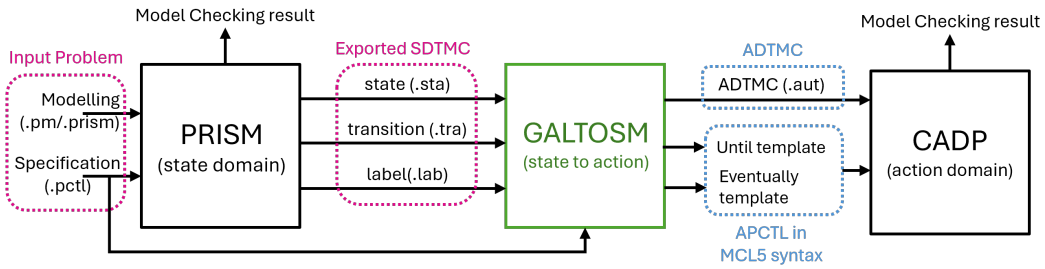
1.2 mCRL2 to STORM/PRISM

To model check the specification of mcrl2 using state based model checking engines, the below workflow need to be followed, where the process starts by specifying the problem using the specification language mcrl2 saved as a .mcrl2 file. Further this file is compiled to build the model using the mCRL2 command mcrl22lps which generated a .lps file using t1 time. Next lps2lts command of mCRL2 converts the generated .lps to .aut using t2 time. The obtained .aut file has a different syntax than the .aur generated by CADP. This .aut file then goes as input to the GALTOSM tool and can output the machine readable formats, .tra and .lab files depending upon the choice you provide during execution. We capture the time required to generate PRISM files by t3 and STORM by t4. Further the generated files can be imported into the respective tools and model check over the PCTL formulas. The model checking time of STORM is captured by t5. A similar approch is used to model check using the PRISM tool.



1.3 PRISM/STORM to CADP

PRISM/STORM takes as input the probabilistic behavior specified in a *.prism/.pm* file for model construction. PRISM can generate a model and export it in the form of Machine Readable Files, consisting of three files, *.sta*, which stores state information (values of the state variables); *.tra*, which stores the probabilities of transitions between states; and *.lab*, which stores the labels of states. Alternatively, a PRISM model can be constructed by manually creating these three files. Next these three files are given as input to our tool GALTOSM to gernerate the .aut file compatible with the CADP toolset. Along with the ADTMC model, it also exports some nescessary data required for MCL formula generation.



2 Grammar of Logics

This section provides a detailed explanation on how to write syntactically correct formulas using the logics discussed in the paper, emphasising on proper bracketing and the modified syntax used for AP(R)CTL and APCTL*. Since these logics share many structural elements, we only explain the APCTL syntax in detail. For the other logics, we only highlight the differences and necessary modifications required for each of them. A set of grammar rules is also provided to help the user generate valid formulas.
Throughout the section, state formulas are represented by ϕ, and ψ has been used to represent either the run or path formula based on the context of the underlying grammar. Run(s) and Path(s) have been used interchangeably, representing the same formula. χ represents a valid auxiliary logic formula as defined in .

2.0.1 APCTL

✔ Grammar:

ϕ ::= `(true)` | `(false)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= `X_`(χ)ϕ | `X_tau`ϕ | ϕ`_`(χ)`U`ϕ | ϕ`_`(χ)`U_`(χ)ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | (χ&χ) | (χ | χ)

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `a` is an action name, which has to be enclosed within "double quotes"

✔ Details:

1. All state formulas(ϕ) are enclosed within `()`.
   Example `(true)`, `(P=?[path formula])`
2. (!ϕ) is the unary boolean NOT operator followed by the operand which is represented in literature by the symbol ¬. The syntax of the formula is `(!(state formula))`.
   Note: As the whole formula is a state formula, there will be `()` around the whole formula, and after `!` there is a state formula so there will also be `()` too.
   Example `(!(true))`, `(!(P=?[path formula]))`

3. The $(\phi\&\phi)$ operator is represented by the symbol &, and $(\phi \mid \phi)$ by |. The syntax of the formula is `((state formula)&(state formula))` and `((state formula)|(state formula))` respectively.
   Note: As the whole formula is a state formula, there will be `()` around the whole formula, and on either side of `&`/| there are state formulas so there will also be `()`.
   Example `((true)&(true))`, `((true)|(P=?[path formula]))`
4. The $(P\lambda x[\psi])$ operator is represented by the symbol `P=?[path formula]`, `P>=0.36[path formula]`, or `P>.6[path formula]`, with similar forms for the less-than comparisons.
   Note: As the whole formula is a path formula, there will be `()` around the whole formula. The formula whose probability we want to calculate will be enclosed by `[]`. *Example* `(P=?[X_("action")(true)])`.
5. Run/path formulas are **not** enclosed in `()`.
   Example `X_tau(state formula)`, `(state formula)_(chi)U(state formula)`
6. X_(χ) operator is represented by `X_(chi)(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). As this operator works on a state formula, the state formula will be enclosed by `()`. And χ will be enclosed within (),
   Example `(P>=[X_(“act1” | “act2”)(true)])`
7. `X_tau` operator is represented by `X_tau(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). As this operator works on a state formula, the state formula will be enclosed by `()`.
   Example `(P>=[X_tau(true)])`
8. ϕ_(χ)Uϕ and ϕ_(χ)U_$(\chi)\phi$ operator is represented by `U` and the format is `(state formula)_(chi)U(state formula)` and `U_(chi)` with format `(state formula)_(chi)U_(chi)(state formula)`.
   Note: On either side of the operator, there are state formulas, so they will be enclosed in `()` and the chi's are enclosed in `()` *Example* `(P=?[(true)_("ACTION")U(true)])`, `(P=?[(true)_("ACTION1")U_("ACTION2")`

2.0.2 APCTL*

✔ Grammar:

ϕ ::= `(true)` | `(false)` | $(!\phi)$ | $(\phi\&\phi)$ | $(\phi \mid \phi)$ | $(P\lambda x[\psi])$

ψ ::= ϕ | $(!\psi)$ | $(\psi\&\psi)$ | $(\psi \mid \psi)$ | $(\mathtt{X}\psi)$ | $(\mathtt{X_}(\chi)\psi)$ | $(\psi\mathtt{U}\psi)$

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | $(\chi\&\chi)$ | $(\chi \mid \chi)$

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `a` is an action name, which has to be enclosed within “double quotes”

✔ Details:
The rules for the state formulas are common since state formula operators are the same as those in APCTL, all operators has been described below

1. $(!\phi), (\phi\&\phi), (\phi \mid \phi), (P\lambda x[\psi])$ has been described in 1,2,3,4 respectively in section 2.0.1.

2. The run/path formulas (ψ) are enclosed in `()` too, contrary to rule 5 of APCTL grammar.
   Note: Rules for the state formula remains the same as 1, which will be enclosed by `()`.
   Example `(P=?[(X_tau(true))])`, `(P=?[((true)&(true))])`

3. ($!\psi$) operator has the same rules as state not formula. The operator representation is `(!(path formula)`.
   Note: The `()` is for the path/run formula where as the `()` is for the state formula.
   Example `(!(true))`

4. ($\psi\&\psi$) and($\psi \mid \psi$) has the same explanation as rule 3 of 2.0.1
   Note: The `()` is for the path/run formula where as the `()` is for the state formula. *Example* `(P=?[((!(true))&(true))])`

5. `X`(ψ) operator explanation is same as `X_tau` and can be found in rule 7 of 2.0.1
   The `()` is for the path/run formula whereas the `()` is for the state formula.
   Example `(P=?[(X(true))])`

6. (`X_`(χ)ψ) operator explanation can be found in rule 6 of 2.0.1
   The `()` is for the path/run formula whereas the `()` is for the state formula.
   Example `(P=?[(X_tau(true))])`

7. (ψ`U`ψ) operator is represented by `U` and the format is `(state formula)U(state formula)`.
   Note: On either side of the operator, there are state formulas, so they will be enclosed in `()`.
   Example `(P=?[(true)U(true)])`.

2.0.3 APRCTL

✔ Grammar:

ϕ ::= `(true)` | `(false)` | ($!\phi$) | ($\phi\&\phi$) | ($\phi \mid \phi$) | ($P\lambda x[\psi]$) | ($E\lambda y[\phi]$)

ψ ::= `X_`(χ)ϕ | `X_tau`ϕ | ϕ`_`(χ)`U`ϕ | ϕ`_`(χ)`U_`(χ)ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

χ ::= `true` | `false` | `!`χ | `"a"` | ($\chi\&\chi$) | ($\chi \mid \chi$)

✔ Note:

- `x`$\in$[0,1] if λ is not `=?`.
- `y` $\in \mathbb{N}$
- `a` is an action name, which has to be enclosed within "double quotes"

✔ Details:
All state and run/path formulas coincide with the APCTL rule, which can be found in 2.0.1 except for the ($E\lambda y[\phi]$), which is explained below.

1. The ($R\lambda y[\phi]$) operator is represented by `E=?[]` or `E>=24[state formula]` or `E>64[state formula]` symbols and same for less than.
   Note: Since E operator has a state formula, `()` as per rule 1 of 2.0.1 inside the [] of E.
   Example `(E=?[(true)])`, `(E=?[(P>=0.5[X_tau(true)])])`

2.0.4 PCTL

✔ Grammar:

ϕ ::= `(true)` | `(false)` | `(ap)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= `X`ϕ | ϕ`U`ϕ

λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
The rules are the same as specified in 2.0.1 point 1,2,3,4,5.

1. All state formulas(ϕ) are enclosed within `()`.
   Example `(true)`, `(P=?[path formula])`
2. `(ap)` is an atomic proposition and is a state formula, so put it in `()` *Example* `("DEAD")`
3. The !ϕ operator is same as rule 2 of 2.0.1
4. The (ϕ&ϕ) operator and (ϕ | ϕ) operator is same as rule 3 of 2.0.1
5. The ($P\lambda x[\psi]$) operator is same as rule 4 of 2.0.1.
   Note: As the whole formula is a path formula, there will be `()` around the whole formula. The formula whose probability we want to calculate will be enclosed by `[]`.
   Example `(P=?[X(true)])`.
6. Run/path formulas are **not** enclosed in `()`.
   Example `X_tau(state formula)`, `(state formula)_(chi)U(state formula)`
7. `X`ϕ operator is represented by `X(state formula)`.
   Note: There will be `no ()` around the whole formula(rule 5). And as this operator works on a state formula, the state formula will be enclosed by `()`.
   Example `(P>=[X(true)])`
8. ϕ`U`ϕ is represented by `(state formula)U(state formula)`.
   Note: There will be no `(` around the whole formula but the state formulas in either sides of the `U` operator will be enclosed by `()`
   Example `(P=?[(true)U(true)])`

2.0.5 PCTL*

✔ Grammar:

ϕ ::= `(true)` | `(false)` | `(ap)` | (!ϕ) | (ϕ&ϕ) | (ϕ | ϕ) | ($P\lambda x[\psi]$)

ψ ::= ϕ | (!ψ) | (ψ&ψ) | (ψ | ψ) | (`X`ψ) | (ψ`U`ψ)

λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
Since the operators of PCTL* is a subset of APCTL*, the rules are the same as 2.0.2

2.0.6 PRCTL

✔ Grammar:

> ϕ ::= `(true)` | `(false)` | `(ap)` | $(!\phi)$ | $(\phi \& \phi)$ | $(\phi \mid \phi)$ | $(P\lambda x[\psi])$ | $(R\lambda y[\phi])$
>
> ψ ::= `X`ϕ | ϕ`U`ϕ | $\phi \leq rU\phi$
>
> λ ::= `=?` | `>=` | `>` | `<=` | `<`

✔ Note:

- `x`∈[0,1] if λ is not `=?`.
- `r,y` $\in \mathbb{N}$
- `ap` is an atomic proposition, which has to be enclosed within "double quotes"

✔ Details:
All state operators are common to the PCTL state operators in section 2.0.4 except the $(R\lambda y[\phi])$. The path/run formula of PRCTL has one additional operator from PCTL, which is $\phi \leq rU\phi$, they are explained below:

1. The $(R\lambda y[\phi])$ operator is represented by `R=?[]` or `R>=24[state formula]` or `R>64[state formula]` symbols and same for less than.
   Note: Since R operator has a state formula, `()` as per rule 1 of 2.0.1 inside the [] of E.
   Example `(R=?[(P>=1[X(true)])])`
2. $\phi \leq rU\phi$ operator is represented in the format `(state formula)<=rU(state formula)`. The rules of the until operator as defined in rule 8 in section 2.0.4
   Example `(R=?[(P>=0.1[("act1")<=25U("act2")])])`